Introduction to Embedded Systems

Course Code: 244066

Lab 3

STM32 Serial Communication Interface

By

Dr. Muhammad Imran Nadeem

Zhengzhou University

Instructions

- Complete every task in the lab practice session.
- For each task, include a 2–5 line description along with a screenshot of both your code and its output.
- Submit your lab report by email to Imran.Nadeem@outlook.com, using the subject line format as:
- **ES24-[lab number]-[registration number]**.
- Lab report need to be submitted before **11:59 pm**.
- Submissions received after the deadline will not be accepted.

Overview

Overview of Learning Outcomes

By completing these simulated labs, you will:

- Understand USART initialization and transmission using polling.
- Observe serial output using a simulator.
- Understand USART receive (RX) process.
- Learn to use polling for two-way serial communication.
- Understand how to integrate standard C I/O functions with USART.
- Learn to use serial console as a debugging tool.

Setup Notes

- Use the same project template as previous labs: device **STM32F103C4**, CMSIS Core + Device Startup selected, #**include "stm32f10x.h**", Debug tab set to **Use Simulator**, and the same **STMaccess.ini** file so the simulator allows access to peripheral registers (**0x40000000–0x400FFFFF**).
- Verify results in the Watch window by adding expressions such as **USART1->CR1, USART1->SR**, **USART1->DR** and **GPIOA->CRH**. Set the display to **Hex**.
- If your µVision build shows **View → Serial Windows → UART #1**, use it to see transmitted text and to type received characters.
- The simulator may not set USART status flags by itself. If a **while** wait loop hangs, edit the value in the Watch window. Set **USART1->SR to 0x000000C0** (TXE and TC) for transmit tasks, **or to 0x00000020** (RXNE) for receive tasks.
- Treat the system clock as **72 MHz**.

Task 1: Enable Clocks and Configure the USART1 Pins

Steps

1. Create a new project and add a main.c file.
2. Enable the GPIOA and USART1 clocks: **RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;** and **RCC->APB2ENR |= RCC_APB2ENR_USART1EN**;.
3. Clear the configuration fields for PA9 and PA10 in **GPIOA->CRH**.
4. Configure PA9 (TX) as Alternate Function push-pull, 50 MHz (field value **0xB**). Configure PA10 (RX) as floating input (field value **0x4**).
5. Build the project and start the debug session.
6. Add **RCC->APB2ENR** and **GPIOA->CRH** to the Watch window.
7. Verify that bits 2 and 14 of **RCC->APB2ENR** are 1, that PA9's 4-bit field in **GPIOA->CRH** reads **0xB**, and that PA10's field reads **0x4**.

Task 2: Configure the 8N1 Frame and Enable USART1

Steps

1. Reuse your clock and pin setup from Task 1 and set the baud rate (9600).
2. Configure the 8N1 frame by clearing the **M** bit in CR1 (8 data bits), clearing **PCE** in CR1 (no parity), and clearing the STOP bits in CR2 (1 stop bit).
3. Enable the transmitter, receiver and USART with TE, RE and UE in **USART1->CR1**.
4. Put all of this inside a function called **USART1_Init()** and call it from main(), followed by an empty while(1) loop.
5. Add **USART1->CR1**, **USART1->CR2** and **USART1->BRR** to the Watch window.
6. Verify that **USART1->CR1** reads **0x0000200C** (UE, TE and RE set).
7. Clear the UE bit manually in the Watch window, then set it again. Note which bit changed in the value.

Task 3: Transmit a String

Steps

1. Write **USART1_SendString(const char *s)** that loops through the string and calls **USART1_SendChar()** for each character until the null terminator.
2. After the last character, make the function wait for the **TC** flag so the final byte has fully left the shift register.
3. In main(), send "**Hello STM32\r\n**" once, then loop forever.
4. Set a breakpoint inside the loop of **USART1_SendString()** and add **USART1->DR** to the Watch window.
5. Step through several iterations and write down the **hex** values that appear in **DR**.
6. Compare them with the ASCII codes of the first few letters of the string.
7. If the UART #1 window is available, check that the full message appears there.

Task 4: Receive a Character and Echo It Back

Steps

1. Write **char USART1_ReceiveChar(void)** that waits until RXNE in **USART1->SR** is 1, then returns **USART1->DR**.
2. In main(), create an infinite loop that receives one character and immediately sends it back with **USART1_SendChar().**
3. Add **USART1->SR**, **USART1->DR** and a variable received to the Watch window.
4. Start the debug session. Type a character in the UART #1 window if it is available.
5. If it is not available, simulate the incoming data by writing **0x41** into **USART1->DR** and **0x00000020** into **USART1->SR** in the Watch window, then run.
6. Confirm that received matches the character you sent and that it is echoed through **DR** again.
7. Repeat with at least three different characters.

Task 5: Interrupt-Driven Reception with RXNE

Steps

1. Extend **USART1_Init()** to enable the receive interrupt by setting **RXNEIE** (bit 5) in **USART1->CR1**.
2. In main(), call **NVIC_SetPriority(USART1_IRQn, 1)**; and **NVIC_EnableIRQ(USART1_IRQn);**
3. Declare two global variables: volatile char **last**_char; and volatile uint32_t **rx_count**;.
4. Write void **USART1_IRQHandler(void)** that checks the **RXNE** flag, reads **USART1->DR** into **last_char**, increments **rx_count**, and echoes the character back.
5. In the main loop, do not poll the USART. Increment a counter variable **idle_loops** to show the CPU is free to do other work.
6. Set a breakpoint inside the ISR and add **last_char**, **rx_count**, **idle_loops** and **USART1->SR** to the Watch window.
7. Trigger the interrupt by typing in the UART #1 window if available. If the interrupt does not fire, set **RXNE** in **USART1->SR** and pend the interrupt manually with **NVIC->ISPR[1] = 0x20** in the Watch window (USART1 is IRQ 37).
8. Check that execution stops in the ISR, that **rx_count** increases by one per character, and that **idle_loops** keeps increasing between interrupts.